Virtual Maize Field Generator — Complete Reference

Parameters, Flowcharts, Available Models & Dynamic Fields

April 2026

Contents

1. Overview **2**

2. Parameter Priority Table **3**

3. Available 3D Models **5**
- 3.1 Crop Models . . . 5
- 3.2 Weed Models . . . 5
- 3.3 Litter Models . . . 5
- 3.4 Obstacle Models (Not configurable via YAML) . . . 5
- 3.5 Marker Models . . . 5

4. Row Generation Flowchart **6**

5. World Generation Pipeline **7**

6. Can Fields Be Dynamic? **8**
- What is Static . . . 8
- How to Simulate "Dynamic" Fields . . . 8
- Adding New Crop/Weed/Litter Models . . . 8

7. Minimal Config (Copy-Paste Starter) **9**

8. Template Configs by Use Case **9**
- 8.1 Perception Testing (dense, tall, no gaps) . . . 9
- 8.2 Navigation Challenge (curves + obstacles) . . . 9
- 8.3 Quick Debug (tiny field, fast loading) . . . 9
- 8.4 Weed Detection . . . 10
- 8.5 Litter/Object Detection . . . 10

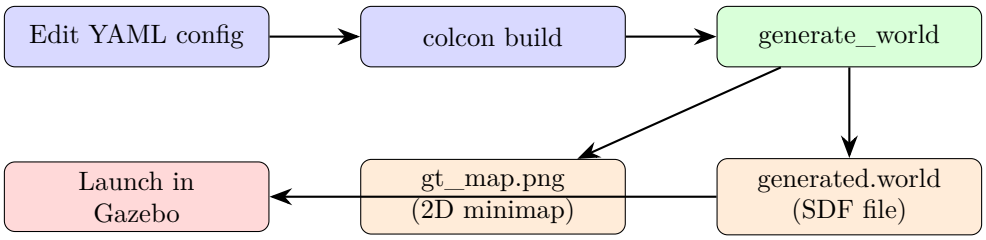
9. Output Files Reference **11**

10. Quick Command Reference **11**

1. Overview

The field generator creates randomized corn (maize) field worlds for Gazebo simulation. Each field is procedurally generated from a YAML config file with controllable randomness.

Key workflow:



Commands:

```
# 1. Edit config
nano src/virtual_maize_field/config/my_field.yaml

# 2. Rebuild (required after editing)
colcon build && source install/setup.bash

# 3. Generate
ros2 run virtual_maize_field generate_world my_field

# 4. Launch
ros2 launch virtual_maize_field simulation.launch.py
```

2. Parameter Priority Table

Color key: REQUIRED = you almost always need to set this, RECOMMENDED = set for most use cases, OPTIONAL = only for specific scenarios.

Parameter	Priority	Default	Type	What it does
FIELD LAYOUT				
`row_length`	Must set	12.0	float	Length of each crop row (m)
`rows_count`	Must set	6	int	Number of parallel rows
`row_width`	Recommer	0.75	float	Gap between rows (m)
`row_segments`	Must set	[straight, curved]	list	Row shape types to use
`rows_curve_budget`	Optional	1.5708	float	Max total curvature (rad)
STRAIGHT SEGMENTS				
`row_segment_straight_le`	Optional	0.5	float	Min straight length (m)
`row_segment_straight_le`	Optional	1.0	float	Max straight length (m)
SINUSOIDAL CURVES				
`row_segment_sincurved_o`	Optional	0.5	float	Min lateral offset (m)
`row_segment_sincurved_o`	Optional	1.5	float	Max lateral offset (m)
`row_segment_sincurved_l`	Optional	3.0	float	Min S-curve length (m)
`row_segment_sincurved_l`	Optional	5.0	float	Max S-curve length (m)
CURVED SEGMENTS				
`row_segment_curved_radi`	Optional	3.0	float	Min turn radius (m)
`row_segment_curved_radi`	Optional	10.0	float	Max turn radius (m)
`row_segment_curved_arc_`	Optional	0.3	float	Min arc angle (rad)
`row_segment_curved_arc_`	Optional	1.0	float	Max arc angle (rad)
ISLAND SEGMENTS				
`row_segment_island_radi`	Optional	1.0	float	Min island radius (m)
`row_segment_island_radi`	Optional	3.0	float	Max island radius (m)
GROUND / TERRAIN				
`ground_elevation_max`	Recommer	0.2	float	Terrain bumpiness (m). Set 0 for flat.
`ground_resolution`	Optional	0.02	float	Heightmap detail (m/pixel)
`ground_headland`	Optional	2.0	float	Flat turning area around field (m)
`ground_ditch_depth`	Optional	0.3	float	Ditch depth at edges (m)
PLANT PROPERTIES				
`plant_height_min`	Recommer	0.3	float	Shortest corn height (m)
`plant_height_max`	Recommer	0.6	float	Tallest corn height (m)
`plant_spacing_min`	Optional	0.13	float	Min in-row spacing (m)
`plant_spacing_max`	Optional	0.19	float	Max in-row spacing (m)

Parameter	Priority	Default	Type	What it does
`plant_radius`	Optional	0.3	float	Collision radius (m)
`plant_radius_noise`	Optional	0.05	float	Radius variation (m)
`plant_placement_error_m`	Optional	0.02	float	Planting error (m)
`plant_mass`	Optional	0.3	float	Plant mass (kg)
GAPS / HOLES				
`hole_prob`	Recommer	[0.06,...]	float/lis	Gap probability (0–1) per row
`hole_size_max`	Recommer	[7,...]	int/list	Max missing plants per gap
CROP TYPES				
`crop_types`	Optional	[maize_01, maize_02]	list	Which corn models to use
WEEDS				
`weeds`	Optional	0	int	Number of weed objects
`weed_types`	Optional	[nettle, unknown_wee dandelion]	list	Weed models
LITTER / OBSTACLES				
`litters`	Optional	0	int	Number of litter objects
`litter_types`	Optional	[ale, beer, coke_can, retro_pepsi_	list	Litter models
SPECIAL				
`ghost_objects`	Optional	false	bool	No-collision objects (vision only)
`location_markers`	Optional	false	bool	Place A/B navigation markers
REPRODUCIBILITY				
`seed`	Must set	-1	int	Random seed. -1 = random, fixed number = reproducible

3. Available 3D Models

3.1 Crop Models

Model Name	Type	Notes
`maize_01`	Corn plant variant 1	Default mesh, randomly scaled per config
`maize_02`	Corn plant variant 2	Alternate mesh, mixed with maize_01

Only 2 crop models are available. Both are maize (corn). Other crop types would require adding new 3D model folders to `models/`.

3.2 Weed Models

Model Name	Config Key	Available	Notes
`nettle`	`nettle`	Yes	Stinging nettle mesh
`unknown_weed`	`unknown_weed`	Yes	Generic weed mesh
`dandelion_*`	`dandelion`	**NO**	Regex-matched, models not shipped

Warning: The `dandelion` weed type is defined in code but the actual 3D models (`dandelion_01`, `dandelion_02`, etc.) are **not included** in the repository. They were only distributed to FRE 2022 competition participants. Using `dandelion` in your config will cause an error. **Only use `nettle` and `unknown_weed`.**

3.3 Litter Models

Model Name	Config Key	Notes
`ale`	`ale`	Ale bottle, placed on its side
`beer`	`beer`	Beer bottle, placed on its side
`coke_can`	`coke_can`	Coca-Cola can, slightly raised
`retro_pepsi_can`	`retro_pepsi_can`	Retro Pepsi can, slightly raised

3.4 Obstacle Models (Not configurable via YAML)

Model Name	Notes
`stone_01`	Small stone
`stone_02`	Large stone
`box`	Generic box obstacle

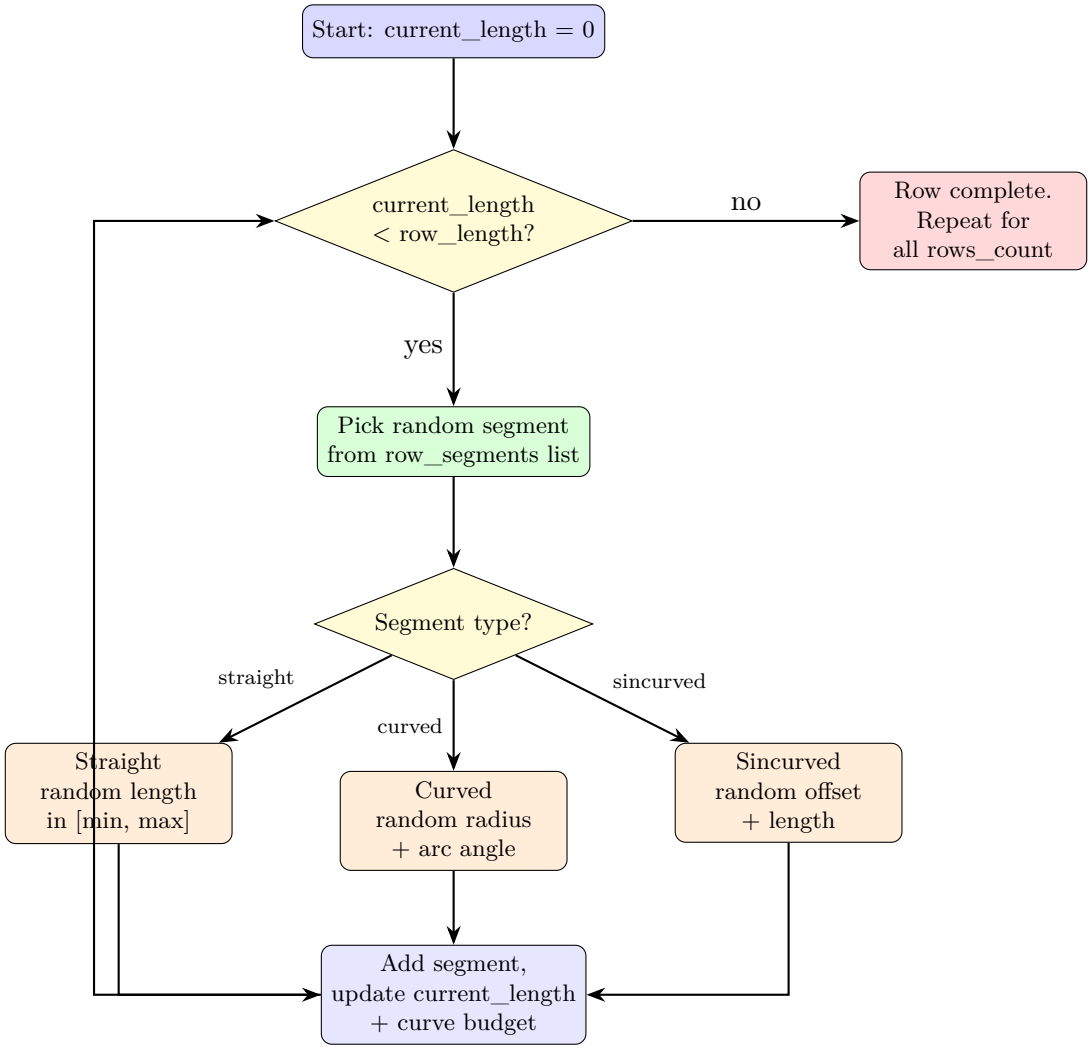
These are defined in code but not exposed as YAML parameters. They could be added with code modification.

3.5 Marker Models

Model Name	Enabled by
`location_marker_a`	`location_markers: true`
`location_marker_b`	`location_markers: true`

4. Row Generation Flowchart

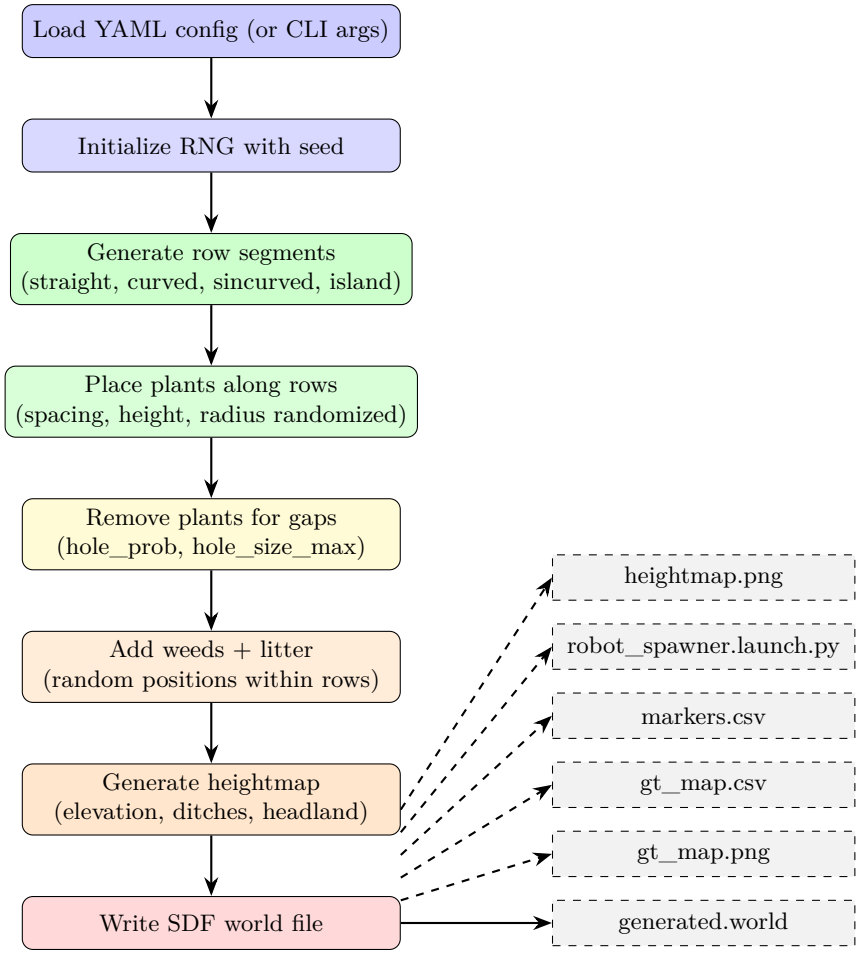
The generator builds each row by chaining random segments until the target `row_length` is reached.



After all rows are generated, the generator:

1. Applies bounded Gaussian noise to plant placements
2. Removes plants to create holes/gaps based on `hole_prob`
3. Scatters weeds randomly within the rows
4. Places litter objects randomly in the field
5. Generates terrain heightmap with elevation and ditches
6. Saves the SDF world file

5. World Generation Pipeline



6. Can Fields Be Dynamic?

Short answer: No. The field is **static** once generated. Plants do not grow, move, or change during simulation.

What is Static

Aspect	Behavior
Plant positions	Fixed at generation time
Plant heights	Fixed at generation time
Terrain	Fixed heightmap
Weeds/litter	Fixed positions
Weather/lighting	Fixed (Gazebo default)

How to Simulate "Dynamic" Fields

You can approximate dynamic behavior with these workarounds:

Goal	How to achieve
Different field each run	Set `seed: -1` (uses timestamp, unique every time)
Growth stages	Create multiple configs with increasing `plant_height_min/max`. Run them in sequence.
Seasonal variation	Change `plant_height`, `hole_prob`, and `weeds` between configs
Field damage	Increase `hole_prob` and `hole_size_max`
Weather variation	Modify Gazebo world `<scene>` lighting after generation
Multiple trials	Use different fixed seeds: `seed: 1`, `seed: 2`, etc.
Runtime plant removal	Use Ignition's `/world/.../remove` service to delete plant entities during simulation
Runtime object spawning	Use Ignition's `/world/.../create` service to add objects during simulation

Adding New Crop/Weed/Litter Models

To add a custom 3D model:

1. Create a folder: `models/my_plant/`
2. Add: `model.config`, `model.sdf`, and mesh files
3. Register in `world_generator/models.py`:

```
CROP_MODELS = {
    "maize_01": GazeboModel(model_name="maize_01"),
    "maize_02": GazeboModel(model_name="maize_02"),
    "my_plant": GazeboModel(model_name="my_plant"),  # <-- add here
}
```

4. Use in config: `crop_types: [maize_01, my_plant]`

7. Minimal Config (Copy-Paste Starter)

You only need 4–5 lines for a working config. Everything else uses defaults.

```yaml
# my_field.yaml - minimal working config
row_length: 10.0
rows_count: 6
row_segments:
  - straight
seed: 42
```

8. Template Configs by Use Case

8.1 Perception Testing (dense, tall, no gaps)

```yaml
row_length: 12.0
rows_count: 8
row_width: 0.60
row_segments:
  - straight
plant_height_min: 0.5
plant_height_max: 0.8
plant_spacing_min: 0.10
plant_spacing_max: 0.14
hole_prob: 0.0
hole_size_max: 0
ground_elevation_max: 0.0
seed: 100
```

8.2 Navigation Challenge (curves + obstacles)

```yaml
row_length: 15.0
rows_count: 10
row_segments:
  - straight
  - curved
  - sincurved
rows_curve_budget: 2.0
weeds: 10
litters: 5
location_markers: true
ground_elevation_max: 0.3
seed: 77
```

8.3 Quick Debug (tiny field, fast loading)

```yaml
row_length: 4.0
rows_count: 3
row_segments:
```

```yaml
  - straight
ground_elevation_max: 0.0
hole_prob: 0.0
hole_size_max: 0
seed: 1
```

8.4 Weed Detection

```yaml
row_length: 10.0
rows_count: 6
row_segments:
  - straight
weeds: 20
weed_types:
  - nettle
  - unknown_weed
hole_prob: 0.02
hole_size_max: 3
seed: 55
```

8.5 Litter/Object Detection

```yaml
row_length: 10.0
rows_count: 6
row_segments:
  - straight
litters: 15
litter_types:
  - ale
  - beer
  - coke_can
  - retro_pepsi_can
ghost_objects: true
seed: 88
```

9. Output Files Reference

File	Description
`generated.world`	SDF world file for Ignition Gazebo. Contains all plants, terrain, weeds, litter, markers.
`gt_map.png`	2D bird's-eye view minimap showing plant positions, START location, and markers A/B.
`gt_map.csv`	CSV with columns: X, Y, kind. Every plant, weed, and litter position with its type label.
`markers.csv`	Marker A and B positions (only populated if `location_markers: true`).
`robot_spawner.launch.py`	Auto-generated ROS 2 launch file to spawn a robot at the correct start position and orientation.
`virtual_maize_field_heightmap.png`	Grayscale heightmap image used for terrain generation.
`driving_pattern.txt`	Text description of the expected robot driving pattern (e.g., "S – 1L – 1R – ...")

10. Quick Command Reference

Command	What it does
`ros2 run virtual_maize_field generate_world`	Generate with defaults
`ros2 run virtual_maize_field generate_world my_field`	Generate from config file
`ros2 run virtual_maize_field generate_world -rows_count 10`	Override single parameter
`ros2 run virtual_maize_field generate_world my_field -show_map`	Generate and display 2D map
`ros2 run virtual_maize_field generate_world -help`	Show all parameters
`./run_field.sh my_field`	Convenience: generate + launch
`./run_field.sh -help`	List all available configs